AutoCareHub — Registro de Mejoras

> *Documento vivo. Cada vez que se añade una funcionalidad nueva se registra aquí con una breve explicación, los archivos implicados y, si aplica, los cambios en la base de datos.*

Proyecto: AutoCareHub (Angular + PHP/MySQL)

Autor: Nicolás García Moreira

Última actualización: 16/10/2026

> ***Restricción del proyecto:** la API existente no se modifica. Las funcionalidades nuevas se construyen creando **llamadas nuevas**, nunca editando las llamadas ya existentes.*

Índice

1. Agenda Inteligente
2. Agenda Semanal del taller (calendario)
3. Gestión del ciclo de vida de las citas
4. Vista «Mis Citas» del cliente
5. Generador de facturas más intuitivo
6. VeriFactu (AEAT)
7. Seguridad
8. Cambios en la base de datos
9. Despliegue automático a InfinityFree
10. Próximas mejoras propuestas

---

1. Agenda Inteligente

Qué es: sustituye el sistema antiguo de citas (horario fijo 9–18, L–V, huecos de 1 hora y capacidad implícita de 1 coche) por una agenda configurable y realista.

Qué aporta:

- **Configuración por taller**: para cada día de la semana se define si abre, su hora de apertura/cierre, un descanso opcional, la **capacidad** (número de coches que puede atender a la vez / bahías) y la **granularidad** de la rejilla de huecos.
- **Catálogo de servicios con duración**: cada servicio (p. ej. «Cambio de aceite, 30 min») tiene una duración estimada y un precio opcional.
- **Disponibilidad real**: al pedir cita, el cliente elige un servicio y el sistema

calcula los huecos realmente libres teniendo en cuenta el horario, el descanso, la duración del servicio y los **solapamientos** con otras citas, respetando la capacidad.

- **Reserva segura**: la creación de la cita valida el horario y comprueba la capacidad dentro de una **transacción**, evitando dobles reservas.

Explicación técnica: la disponibilidad se calcula recorriendo cada día del rango, generando huecos desde la apertura hasta el cierre en pasos de `SlotMinutes`, y descartando los que pisan el descanso o ya pasaron. Para cada hueco se cuentan las citas que se solapan (`inicio < finHueco AND fin > inicioHueco`); si ese número es menor que la capacidad, el hueco está disponible.

Archivos: todo el backend nuevo vive en archivos nuevos, **sin modificar la API original** (`appointments.php` queda intacto). La reserva inteligente es la acción nueva `crear_cita` en `schedule.php`.

- Backend (nuevo): `PHP/models/Schedule.php`, `PHP/controllers/ScheduleController.php`, `PHP/routes/schedule.php`.
- Frontend: `components/agenda-config/` (configuración del taller), `components/make-appointment/` (reserva con servicio y huecos reales).

---

2. Agenda Semanal del taller

Qué es: una vista de **calendario semanal** para el taller, en lugar de un simple listado de citas.

Qué aporta:

- 7 columnas (Lunes–Domingo) con las citas de cada día ordenadas por hora.
- Navegación entre semanas y botón «Hoy».
- Colores por estado y leyenda.
- Acciones rápidas sobre cada cita (ver punto 3).

Archivos: `components/workshop-agenda/`.

---

3. Gestión del ciclo de vida de las citas

Qué es: permite mover una cita por sus estados:

`Pendiente` → `Confirmada` → `Finalizada`, o `Cancelada`.

Qué aporta:

- El **taller** confirma / finaliza / cancela cada cita desde la Agenda Semanal.

- Cuando una cita se **cancela, su hueco se libera automáticamente** (el cálculo de disponibilidad ignora las citas canceladas).
- Validación de permisos: el taller solo puede tocar citas de su propio taller.

Archivos: acciones nuevas `cambiar_estado` y `citas_taller` en `schedule.php` (`ScheduleController` + modelo `Schedule.php`); frontend `components/workshop-agenda/`.

---

4. Vista «Mis Citas» del cliente

Qué es: una pantalla para que el cliente vea y gestione sus propias citas.

Qué aporta:

- Lista de citas con filtro **Próximas / Todas**, mostrando taller, vehículo y servicio.
- Botón **Cancelar** en las citas futuras (libera el hueco para otros clientes).
- Colores por estado.

Archivos: `components/my-appointments/`; acciones nuevas `mis_citas` y `cancelar` en `schedule.php`.

---

5. Generador de facturas más intuitivo

Qué es: se rediseña la forma en la que el taller crea una factura, sin perder la generación del PDF con los datos.

Qué aporta:

- **Cita por desplegable**: se elige la cita de una lista (`#ID · fecha · vehículo · cliente`) en lugar de escribir el ID a mano.
- **Resumen de la cita** seleccionada (estado, vehículo, cliente, servicio).
- **Ítems desde el catálogo de servicios**: un selector autocompleta descripción y precio del servicio; se mantiene la opción de escribir el ítem manualmente.
- El **PDF** se sigue generando igual (jsPDF) con todos los datos de la factura.
- **Descarga de PDF también para el taller**: la lógica del PDF se extrajo a un servicio reutilizable (`services/invoice-pdf.service.ts`) usado tanto por el cliente (*Ver facturas*) como por el taller, que ahora tiene un botón **PDF** en cada factura de su lista. No se modificó ninguna llamada de la API: se reutiliza la existente y los datos que ya devuelve. Se añade el nombre del cliente al PDF cuando está disponible.
- **Borrador de factura automático al finalizar una cita**: en la *Agenda Semanal*,

las citas *Finalizadas* muestran un botón **Facturar** que abre el generador con la cita ya seleccionada y el **servicio realizado añadido como ítem** (descripción y precio tomados del catálogo). El taller solo revisa y confirma. No se toca la API: la precarga usa la acción nueva `citas_taller` (que ahora incluye `ServiceID`) y la navegación se hace con un parámetro `?appointmentId=` en el frontend.

- **PDF de factura rediseñado** (`services/invoice-pdf.service.ts`): se sustituye el volcado de texto sobre una imagen de fondo por un diseño profesional —cabecera con logo, «FACTURA», nº y fecha y **pastilla de estado** con color; bloques separados de **emisor** (con NIF) y **cliente + vehículo**; **tabla** de líneas con cabecera y filas alternas; **desglose Base imponible · IVA · Total** en caja; **importes en formato de moneda español (`49,90 €`); recuadro** VeriFactu con QR**; y pie de página. Solo frontend, no toca la API.
- **Estado «Facturada» en las citas**: como la API existente, al crear una factura, deja la cita en *Finalizada* (no hay un estado «Facturada» y no se puede modificar), ese estado se **deriva** de si la cita tiene factura asociada. Las acciones nuevas `citas_taller` y `mis_citas` devuelven el `InvoiceID` de la cita, y la interfaz muestra una etiqueta **«Facturada»** (en la *Agenda Semanal* del taller y en *Mis Citas* del cliente). En la agenda, una cita ya facturada oculta el botón *Facturar* para no duplicar facturas. No se toca la API original de facturas.

Archivos: `components/invoices-generator/`, `components/invoice-viewer/`, `services/invoice-pdf.service.ts`, `components/workshop-agenda/`, `components/my-appointments/`. El desplegable de citas y el estado «Facturada» usan la acción nueva `citas_taller` de `schedule.php` (servicio + `InvoiceID`), sin tocar la API original.

---

6. VeriFactu (AEAT)

Qué es: implementación del sistema antifraude **VeriFactu** (RD 1007/2023) para los registros de facturación, respetando la restricción de no tocar la API existente (todo en endpoints y tablas nuevos, en solo lectura sobre las facturas).

Qué aporta:

- **Datos fiscales del emisor** (NIF + razón social) configurables por taller, en la página de *Agenda Inteligente*.
- **Registro de facturación con huella encadenada**: al descargar una factura se genera

(de forma idempotente) su registro VeriFactu con una **huella SHA-256** calculada según el orden de campos de la AEAT, que **incorpora la huella del registro anterior** del mismo emisor. Así la secuencia de facturas es a prueba de manipulación.

- **Código QR** de verificación de la AEAT y la marca **«VERI*FACTU»** impresos en el PDF de la factura (tanto para el taller como para el cliente).
- **Verificación de integridad de la cadena**: el taller puede recalcular toda su cadena de huellas y comprobar que no se ha alterado ninguna factura.

Fuera de alcance (producción): el envío en tiempo real al servicio web de la AEAT (requiere certificado digital y alta en Hacienda). Aquí se genera y encadena el registro localmente, que es la parte demostrable.

Archivos (todo nuevo): BD `PHP/migrations/2026_verifactu.sql` (tablas `VerifactuEmisor` y `VerifactuRegistros`); backend `PHP/models/Verifactu.php`, `PHP/controllers/VerifactuController.php`, `PHP/routes/verifactu.php` (acciones `obtener_emisor`, `guardar_emisor`, `generar`, `obtener`, `verificar_cadena`); frontend `services/invoice-pdf.service.ts` (QR + marca), `components/agenda-config/` (datos fiscales + verificación) y dependencia `qrcode`.

---

7. Seguridad

Qué es: primera tanda de mejoras de seguridad, respetando la restricción de no modificar la API existente (solo frontend y endpoints nuevos).

Qué aporta:

- **Guards de ruta en Angular** (`services/auth.guards.ts`): ya no se puede navegar por URL sin haber iniciado sesión, y cada ruta está **restringida por rol** (cliente / taller / administrador). Si no hay sesión redirige a `/login`; si el rol no corresponde, a `/home`. Cualquier ruta desconocida cae en `/login`.
- **Endurecimiento de los endpoints nuevos** (`schedule.php`):
- Al reservar, se comprueba que el **vehículo pertenece al usuario** autenticado (antes se confiaba en el `VehicleID` recibido) → responde 403 si no.
- Una cita nueva **siempre nace en estado** `Pendiente`; se ignora cualquier `Status` enviado por el cliente.

Nota: la autorización de datos la sigue garantizando el backend (sesión PHP); los guards protegen la navegación de la interfaz. Quedan pendientes (requieren tocar la API existente, fuera del alcance actual): credenciales de BD fuera del código, cookies de

sesión `HttpOnly`/`SameSite`, CORS y tokens CSRF.

Archivos: `services/auth.guards.ts`, `app.routes.ts`; backend nuevo `ScheduleController.php` y `models/Schedule.php`.

---

8. Cambios en la base de datos

Migración: `PHP/migrations/2026_agenda_inteligente.sql` (idempotente y aditiva; no borra datos). Resumen:

Cambio	Descripción
Tabla `WorkshopSchedules`	Horario semanal por taller: día, abierto/cerrado, apertura, cierre, descanso, capacidad y granularidad.
Tabla `WorkshopServices`	Catálogo de servicios por taller: nombre, duración, precio, activo.
Columna `Appointments.ServiceID`	Enlaza cada cita con el servicio elegido (FK opcional).
Índice `idx_appt_workshop_time`	Acelera las consultas de disponibilidad.

También se actualizó `PHP/paginaCreacionBase.php` para que una base de datos nueva incluya ya estas tablas y columnas.

---

9. Despliegue automático a InfinityFree

Qué es: cada `push` a la rama `main` publica la web en InfinityFree sin pasos manuales, mediante GitHub Actions.

Cómo funciona (`.github/workflows/deploy-infinityfree.yml`):

1. Compila el frontend Angular en modo producción (usa `environment.prod.ts`).
2. Sube `AUTOCAREHUB/dist/autocarehub/browser/` por FTP a `/htdocs/Autocare/frontend/`.
3. Sube la carpeta `PHP/` a `/htdocs/Autocare/BACKEND/`, **excepto** `config/database.php` (en el servidor tiene los datos de la BD de InfinityFree) y los logs.

Solo se suben los archivos que han cambiado desde el último despliegue. También se puede lanzar a mano desde la pestaña *Actions* de GitHub.

Configuración necesaria: en GitHub › *Settings › Secrets and variables › Actions*, los secretos `FTP_SERVER`, `FTP_USERNAME` y `FTP_PASSWORD` (datos FTP del panel de InfinityFree).

Otros cambios: el `baseHref` de `angular.json` pasa a `/Autocare/frontend/` (en minúsculas, como la carpeta del servidor).

Nota: las migraciones SQL no se ejecutan solas; hay que lanzarlas a mano en la BD

de producción antes de desplegar código que las necesite.

---

10. Próximas mejoras propuestas

Ideas que encajan con lo construido, ordenadas por relación valor / esfuerzo:

Agenda

- **Días festivos / cierres puntuales**: bloquear fechas concretas además del horario semanal.
- **Reprogramar cita**: mover una cita a otro hueco sin cancelar y volver a crear.
- **Lista de espera** cuando un hueco está completo.

Comunicación

- **Recordatorios automáticos** de cita próxima por email/SMS (la tabla `NotificationPreferences` ya existe en el modelo de datos).
- **Reseñas y valoraciones** del taller tras una cita finalizada.

Cliente / negocio

- **Pago online** de facturas (Stripe en modo test) y marcado automático como *Pagada*.
- **Historial de mantenimiento por vehículo** (todas las citas y facturas del coche).
- **Buscador de talleres por cercanía** con mapa.
- **Dashboard de estadísticas** ampliado: ingresos por periodo y exportación.

Calidad y seguridad

- **Recuperar contraseña** por email.
- Endurecer seguridad: tokens CSRF y sacar credenciales de BD del código fuente.
- **Modo oscuro** y mejoras de accesibilidad.

Fin del documento. Añadir las nuevas mejoras al final de la sección correspondiente y actualizar la fecha de la cabecera.